Обучение с частичным привлечением учителя

Как получить качество «полной разметки», когда эксперт поставил метки лишь нескольким процентам объектов

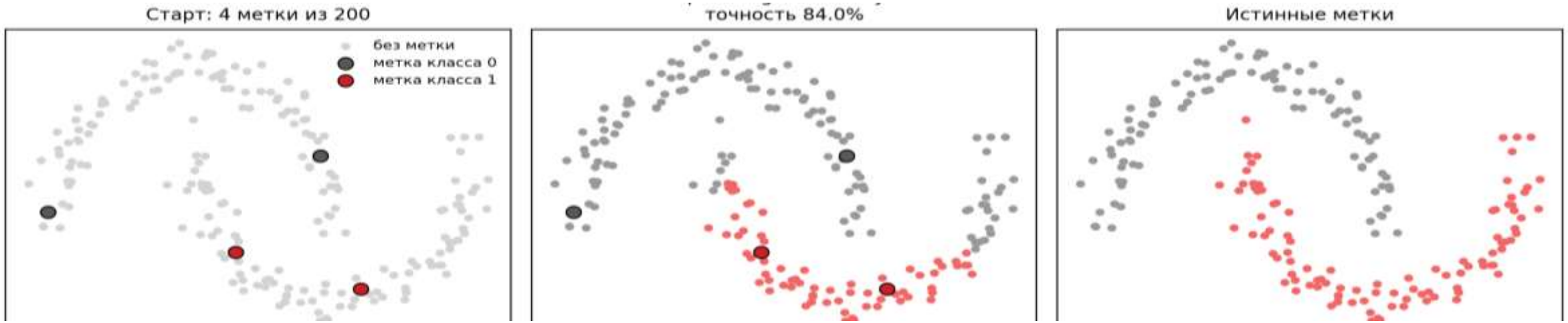


Рис. 0. Четыре метки учителя и 196 неразмеченных объектов: графовый метод восстанавливает два класса (расчёт в src/semi_supervised.py)

Содержание

01 **Зачем это нужно:** цена метки и дефицит разметки 03

02 Постановка задачи и три предположения метода 04

03 Карта подходов: четыре семейства 05

04 Самообучение: цикл, порог уверенности, риски 06

05 **Численный пример** шаг за шагом (две страницы) 07-08

06 Графовые методы: распространение меток 09

07 **Пример решения на Python**: постановка и модели 10-11

08 Результаты эксперимента на рукописных цифрах 12

09 **Альтернативы, сравнение** и выводы 13-14

Меток всегда меньше, чем данных - метод закрывает разрыв

Решаемая задача: обучить модель заданного качества при дефиците меток и избытке сырых объектов

Пр. 1. Узкое место - экспертная разметка, а не сбор данных

Задача	Кто ставит метку	Чем ограничена разметка
Диагноз по снимку МРТ	врач-радиолог	минуты на снимок, высокая ставка
Дефекты на конвейере	инженер-технолог	нужна остановка линии
Тональность отзывов	асессоры	дёшево, но нужны миллионы меток
Разметка аудиозаписей	носители языка	часы прослушивания на час аудио

Иллюстративные оценки трудоёмкости

Когда возникает эта задача

1. **Метка требует эксперта.** Один специалист размечает сотни объектов в день - сканер собирает тысячи

2. **Сырые данные почти бесплатны.** Логи, фото, сигналы датчиков копятся сами собой

3. **Метки дорого обновлять.** При смене правил переразметка всего архива невозможна

Цель метода - качество «как на полной разметке» при малом бюджете на учителя

Итог: метод нужен всюду, где признаки x собираются автоматически, а ответ y — ручным трудом эксперта

Постановка: l размеченных и u ≫ l неразмеченных объектов

Формальная схема задачи и условия, при которых неразмеченные данные информативны

Пр. 2. Обучающая выборка — смесь двух частей

Размеченная часть: $L = \{(x_i, y_i)\}_{i=1}^{l}$

Неразмеченная часть: $U = \{x_j\}_{j=1}^{u}, u \gg l$

Цель — классификатор $f: \mathcal{X} \to \mathcal{Y}$, точнее модели, обученной только на L

Типичные доли: l/u от 1% до 10%. Метки ставит учитель (эксперт, асессор), неразмеченный массив приходит из того же распределения $p(x)$.

Метод трансдуктивен (предсказать метки для U) либо индуктивен (получить f для новых объектов).

Пр. 3. Три предположения - фундамент метода

1. Гладкость. Близким объектам $x \approx x'$ соответствуют одинаковые метки $y = y'$ - плотные области p(x) несут информацию о классах

2. Кластерное предположение. Объекты образуют кластеры; граница решения проходит по областям низкой плотности, а не через кластер

3. Предположение о многообразии. Признаковое пространство избыточно: данные лежат около многообразия меньшей размерности, вдоль которого метка меняется плавно

Проверка перед применением: если кластеры данных не связаны с метками, неразмеченные данные не помогут — метод только добавит шум

Карта методов: четыре семейства решений одной задачи

Карта альтернатив внутри области: что общего у подходов и чем они различаются

Пр. 4. Все четыре семейства используют U, но по-разному связывают его с метками

Семейство	Идея	Представители	Когда выбирать
Обёрточные (wrapper)	Модель обучается на L, ставит псевдометки уверенным объектам из U и дообучается на них	Self-training, Co-training, Tri-training	Любой классификатор; быстрый старт; нейросети
Графовые	Объекты — вершины графа близости; метки «растекаются» по рёбрам до равновесия	Label Propagation, Label Spreading	Чёткая кластерная структура; u умеренно
Генеративные	Совместная модель p(x, y): параметры распределений оцениваются по L и U через EM	GMM + EM, наивный Байес на U	Известна форма распределения p(x|y)
Согласованность (consistency)	Штраф за разные прогнозы на аугментированных копиях одного объекта из U	П-model, Mean Teacher, FixMatch	Изображения и текст; нейросети; есть аугментации

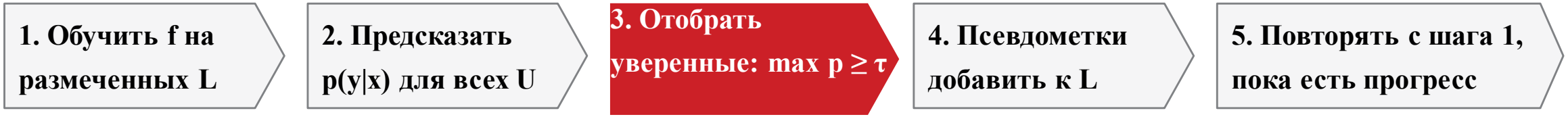
Современные состязательные и self-supervised варианты (SimCLR + тонкая настройка)

Дальше разбираем два базовых подхода — самообучение и распространение меток: они же реализованы в коде

Самообучение: модель сама размечает уверенные прогнозы

Базовый обёрточный алгоритм: любой вероятностный классификатор внутри, снаружи - цикл с порогом

Пр. 5. Один цикл self-training - пять шагов; повторяется до остановки



Критерии остановки: нет объектов выше порога; достигнут лимит итераций; качество на отложенной размеченной выборке перестало расти

Пр. 6. Порог τ - главный регулятор

Низкий τ (0.5-0.7): охват растёт быстро, но ошибочные псевдометки попадают в обучение и усиливаются на следующих итерациях — смещение подтверждения.

Высокий τ (0.9-0.99): псевдометки чище, но их мало, и они концентрируются в «простых» областях - слабый прирост.

Практика: $\tau \approx 0.9$, прирост партиями (top-k на итерацию), контроль на валидации.

Пр. 7. Сильные и слабые стороны

+ Плюсы	− Минусы
Любой базовый классификатор; 5 строк кода	Ошибка ранней итерации каскадируется
Индуктивен: работает на новых данных	Чувствителен к калибровке вероятностей и дисбалансу классов

Самообучение — первая линия: просто, совместимо с любым классификатором, но требует порога и валидации

Численный пример (шаг 1-2): шесть точек и две метки

Одномерная бинарная классификация методом ближайшего центроида; порог отступа $\delta = 2.0$

Дано. Размечены учителем два объекта: **$x = 1 \rightarrow$ класс 0** и **$x = 10 \rightarrow$ класс 1**. Неразмечены: $x = 2, 3, 4, 6, 8, 9$.

Шаг 1. Центроиды по размеченным данным: $c_0 = 1$, $c_1 = 10$; разделяющая граница — середина: $(1 + 10) / 2 = 5.50$.

Шаг 2. Прогноз для неразмеченных точек и отбор по отступу $|d_0 - d_1| \geq \delta$

x	d(x, c_0)	d(x, c_1)	прогноз	отступ	берём?
2	1	8	0	7	**да**
3	2	7	0	5	**да**
4	3	6	0	3	**да**
6	5	4	1	**1**	**нет**
8	7	2	1	5	**да**
9	8	1	1	7	**да**

Точка $x = 6$ прогнозируется классом 1, но её отступ $1.0 < \delta = 2.0$ — она слишком близка к границе и отбрасывается. Порог защищает обучающую выборку от сомнительных псевдометок.

Отобрано пять псевдометок: класс 0 пополнился точками 2, 3, 4; класс 1 — точками 8, 9

Численный пример (шаг 3-4): граница сместилась до 5.75

Пересчёт центроидов по пополненным выборкам и чтение результата итерации

Шаг 3. Новые центроиды

Величина	Точки в выборке	Было → стало
центроид c_0	1; 2; 3; 4	1.00 → **2.50**
центроид c_1	10; 8; 9	10.00 → **9.00**
граница	$(c_0 + c_1) / 2$	5.50 → **5.75**

Шаг 4. Как читать результат

Псевдометки втянули c_0 в плотное облако точек 2-4, поэтому граница сдвинулась на 0.25 вправо - ближе к истинной структуре данных.

Точка x = 6 осталась без метки: на следующей итерации она снова участвует в отборе с новыми центроидами.

Учитель при этом не потратил ни одной новой метки.

Рис. 1. Итерация на числовой прямой: псевдометки и сдвиг границы

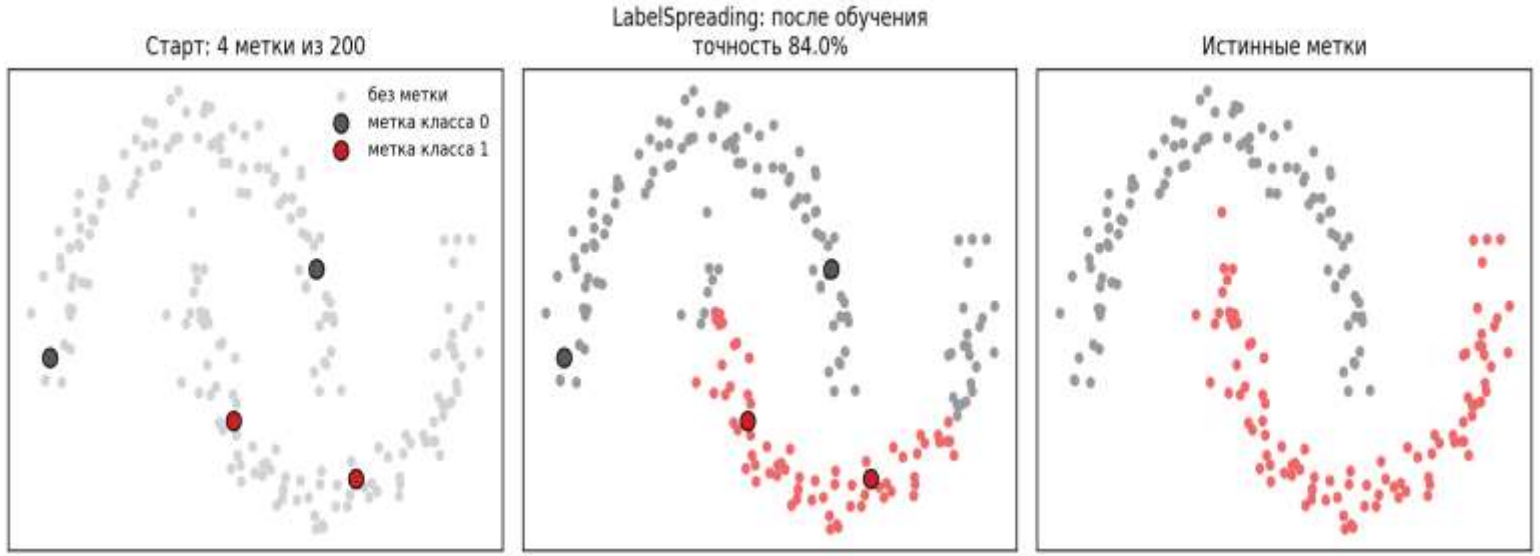


Механика метода: уверенные прогнозы становятся данными — и модель уточняет собственную границу решения

Графовые методы: метки распространяются по рёбрам близости

Label Propagation / Label Spreading: прямая реализация кластерного предположения

Рис. 2. 4 метки из 200 восстанавливают две «ленты» с точностью 84%



Игрушечный датасет two moons (200 объектов),
расчёт: src/semi_supervised.py, часть 3

Пр. 8. Итерация алгоритма

$$Y^{(t+1)} = \alpha\, S\, Y^{(t)} + (1-\alpha)\, Y^{(0)}$$

размеченные строки $Y^{(0)}$ фиксируются

S - нормированная матрица сходства (kNN-граф или RBF-ядро): чем ближе объекты, тем сильнее обмен «верой» в метку.

Сходимость: итерация - это случайное блуждание с поглощением; решение находится за один проход линейной алгебры.

Пределы: стоимость $O(u^2)$ по памяти и чувствительность к «мостам» между кластерами.

Графовые методы сильны там, где кластерная структура выражена — и опасны, когда кластеры связаны шумовыми «мостами»

Эксперимент: 1 797 цифр, метки - лишь у 5% объектов

Постановка эксперимента в коде - фрагмент 1 из кода

Код 1. Данные и маска «учитель разметил 5%»

```
X, y = load_digits(return_X_y=True)   # 1797 объектов 8x8
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)


# учитель ставит метки только 5% обучающей выборки
rng = np.random.RandomState(42)
n_lab = int(len(X_tr) * 0.05)          # 62 объекта
idx = rng.permutation(len(X_tr))[:n_lab]
y_semi = np.full(len(X_tr), -1)  # -1 = метки нет
y_semi[idx] = y_tr[idx]
```

Конвенция sklearn: y = −1 означает «метка отсутствует»; полуучительские модели принимают такую разметку напрямую.

Пр. 9. Протокол эксперимента

Датасет	**digits: 1 797 изображений 8×8, 10 классов**
Train / test	1 257 / 540, стратифицированно
Размечено	62 объекта (5%), выбор случайный, seed 42
Метрика	accuracy на отложенном тесте
Базовая модель	логистическая регрессия
Ориентир	та же модель при 100% меток

Честное сравнение: все три подхода видят одни и те же 62 метки и один и тот же неразмеченный массив.

Вся суть постановки - вектор y_semi: 62 метки учителя и 1 195 значений −1 вместо меток

Три подхода в коде: бейзлайн, самообучение, граф

Фрагмент 2 из кода: каждая модель - три строки, отличия только в использовании U

Код 2. Бейзлайн видит только размеченное - полуучительские модели видят всё

A · Только размеченные 5%

```
base = LogisticRegression(
    max_iter=2000)
base.fit(X_tr[idx],
         y_tr[idx])
acc_base = accuracy_score(
    y_te, base.predict(X_te))
```

U игнорируется: модель видит 62 объекта из 1 257. Это честная нижняя граница качества при таком бюджете разметки.

Б · Самообучение

```
from sklearn.semi_supervised import
SelfTrainingClassifier

self_tr = SelfTrainingClassifier(
    LogisticRegression(max_iter=2000),
    threshold=0.9)   # порог tau
self_tr.fit(X_tr, y_semi)
```

Обёртка над любым классификатором: на 11 итерациях псевдометки получили 1 209 из 1 257 объектов — почти весь U.

В · Распространение меток

```
from sklearn.semi_supervised import
LabelSpreading

spread = LabelSpreading(
    kernel="knn",
    n_neighbors=7)
spread.fit(X_tr, y_semi)
```

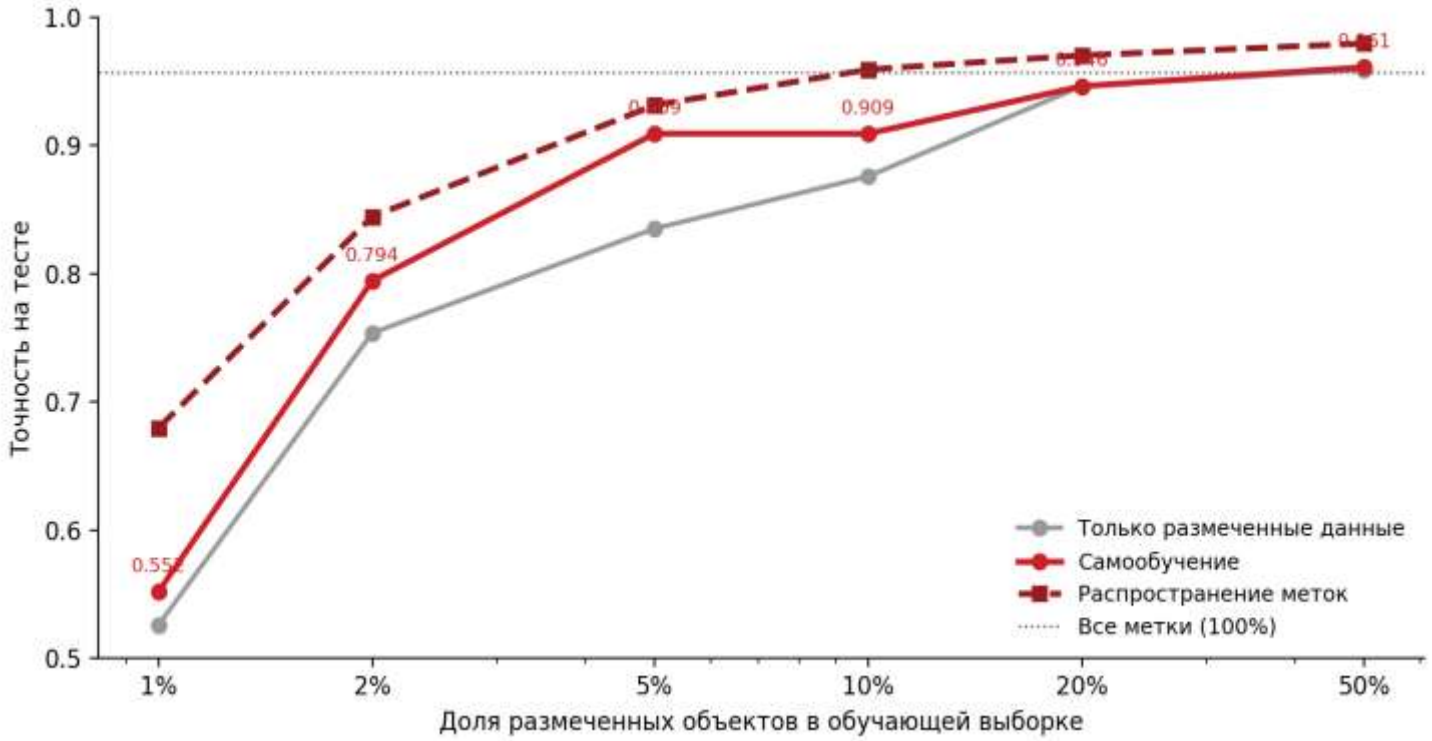
kNN-граф на 7 соседей: метки «растекаются» по рёбрам близости. Трансдуктивно: предсказывает только для данных, которые видел.

Интерфейс одинаковый — fit(X, y_semi): перевести существующий пайплайн на частичного учителя стоит несколько строк

Результат: +7,4 п.п. самообучением, +9,6 п.п. графом

Точность на отложенном тесте (540 объектов) в зависимости от доли размеченных данных, seed 42

Рис. 3. Неразмеченные данные ценнее всего при доле меток 1-10%



Пр. 10. Итог при 5% меток

Подход	Точность	Δ к бейзл.
Только 62 метки	0.835	-
Самообучение	**0.909**	+7.4 п.п.
Распространение	**0.932**	+9.6 п.п.
Все метки (100%)	0.957	+12.2 п.п.

Разрыв с «полной» разметкой - 12.2 п.п.; полуучительские методы закрыли **61-79%** этого разрыва, не потратив ни одной новой метки.

Экономический смысл: качество 0.91+ достигнуто бюджетом разметки в 62 метки вместо 1 257

Альтернативы: чем ещё можно сэкономить на разметке

Сравнение пяти стратегий по механизму экономии, условиям выбора и главному риску

Пр. 11. Стратегии различаются тем, откуда берётся недостающая информация о метках

Подход	Механизм экономии	Когда выбрать	Главный риск
Частичный учитель (наш метод)	Модель сама извлекает метки из структуры U	$u \gg l$, выраженные кластеры, нет бюджета на разметку	Каскад ошибок при нарушении предположений
Активное обучение	Учитель размечает только самые информативные объекты	Эксперт доступен итеративно и недорого	Стоимость циклов; смещение выборки
Слабая разметка (weak supervision)	Метки ставят эвристики и правила, шум усредняется	Есть формализуемые правила предметной области	Систематический шум правил
Трансферное обучение	Модель, обученная на другом домене, дообучается на малых l	Есть близкая предобученная модель	Сдвиг домена, отрицательный перенос
Полная разметка	Учитель размечает всё — экономии нет	Бюджет не ограничен, метки критичны	Дорого, медленно, не масштабируется

Подходы комбинируются: активное обучение выбирает, что разметить; частичный учитель доучивает остальное

Правило выбора: $u \gg l$ и кластерная структура → частичный учитель; дешёвый доступный эксперт → активное обучение

Выводы

1. Задача - дефицит меток при избытке данных. Метод превращает неразмеченный массив U в прирост качества без новых затрат на учителя.

2. Метод работает на предположениях. Гладкость и кластерная структура - условие пользы; при их нарушении псевдометки накапливают ошибку.

3. Самообучение - первая линия. Три строки кода поверх любого классификатора; графовые методы - при чётких кластерах и умеренном u.

4. Эффект измерим. На digits при 5% меток: +7.4 п.п. (самообучение) и +9.6 п.п. (граф) к бейзлайну - закрыто 61-79% разрыва с полной разметкой.